	UNIVERSIDAD AUTÓNOMA DE OCCIDENTE				
	FACULTAD DE INGENIERÍA ***NUCLEO MIDIA***		**ESTRUCTURAS DE DATOS Y ALGORITMOS 1**	***GRUPO***	
	CÓDIGO:		**NOMBRE:**		***Valoración***
TALLER ESTUDIO CORTE 3				**FECHA:**	

PRIMERA PARTE	Obtención de tiempo de ejecución y computo de la complejidad		

Para cada uno de los siguientes algoritmos: calcular la función de recurrencia que define su tiempo de ejecución y resolverla para obtener su complejidad.

a)

```
function recursivo2(A,i,j){
    if (i>j){
        return acota(A)
    }else{
        return Math.max(A[i],A[j]) * recursivo2(A,i+1,j)
    }
}

let info = [4, 7, 21, 10,  ....  , 13, 23]
recursivo2(info, 0, info.length-1)
```

Donde ***acota*** es O(lg n)

b)

```
function recursivo3(A,i,j){
    if (i>j){
        return (A[0]+A[1])/2
    }else{
        let res = obtenerMayor(A)
        let parte = Math.trunc((j+1-i)/3)
        res += recursivo3(A,i,i+parte-1) +
               recursivo3(A,i+parte,i+2*parte-1) +
               recursivo3(A,i+2*parte,j)
        return res
    }
}

let info = [4, 7, 21, 10, ..... , 13, 23]
recursivo3(info, 0, info.length-1)
```

Donde ***obtenerMayor*** es O(n)

c)

Donde ***acota*** es O(lg n)

```
function recursivo4(A,i,j){
    if (i>j){
        return i
    }else{
        let parte = Math.trunc((i+j)/2)
        return recursivo4(A,i,i+parte-1) +
               recursivo4(A,i+parte,j)
    }
}
let info = [4, 7, 21, 10, ..... , 13, 23]
recursivo4(info, 0, info.length-1)
```

SEGUNDA PARTE	Implementación de algoritmos recursivos		

Para cada uno de los siguientes pseudocódigos: determinar que hacen, implementar como funciones de JavaScript, calcular la función de recurrencia que define su tiempo de ejecución y resolverla para obtener su complejidad.

a)

```
PROCEDIMIENTO proceso1 (A,x,p)
   SI p es mayor a tamaño(A)-1
      retornar 0
   SINO
      SI A[p] es igual a x
         retornar 1 + el valor obtenido de invocar a proceso1 con p aumentado en 1
      SINO
         retornar el valor obtenido de invocar a proceso1 con p aumentado en 1

info = [3, 5, 2, 10, 5, ..., 20]
proceso1 (info,5,0)
```

b)

```
PROCEDIMIENTO proceso2 (A,n)
   SI  n es menor o igual a 1
      retornar el valor obtenido de sumar los valores de A
   SINO
      crear variable acum y asignar el valor existente en A en la
      posicion inicial
      acumular en acum el valor obtenido de invocar a proceso2 con
      n que es la mitad del valor inicial
      acumular en acum el doble del valor obtenido de invocar a
      proceso2 con n que es la mitad del valor inicial

      retornar acum


info = [3, 5, 2, 10, 5, ..., 20]
proceso2 (info,tamaño(info))
```

Donde **sumarDatos**(arreglo) es el procedimiento que recorre el arreglo de forma secuencial, para retornar la sumatoria de los valores almacenados.